Kirby e il sito

Beweeg physio

Guida di studio per Amos: come funziona Kirby, come è fatto questo sito e cosa rispondere a chi usa Kirby ogni giorno.

Basata sul codice del progetto `beweeg-physio-kirby` (Kirby 5.6.1) e sulla documentazione ufficiale di getkirby.com, consultata a ottobre 2026.
Quello che non è stato verificato sta nell'ultimo capitolo, "Da verificare". · 26 ottobre 2026

Indice

1. Cos'è Kirby 3

2. Le cartelle del progetto 5

3. Come funziona una richiesta 6

4. Come è fatto questo sito 8

5. Come si fa 11

6. Sicurezza e produzione 13

7. Storia e motivi delle scelte 14

8. Domande che potrebbero farti 15

9. Cose che non so o da verificare 17

1. Cos'è Kirby

Kirby è un CMS in PHP che salva i contenuti in **file di testo e cartelle** invece che in un database. La documentazione lo dice così: "all content is stored in files and folders within the `content` folder". Ogni cartella è una pagina con il suo indirizzo.

getkirby.com/docs/guide/content/introduction

Contenuti: cartelle e file .txt

- **Una cartella = una pagina.** `content/1_chi-sono/` è la pagina `/it/chi-sono`.
- **Il numero davanti al nome** la rende "listed" (pubblicata e ordinata): "Listed pages are published pages with a prepended number by which they can be sorted". Senza numero è "unlisted", raggiungibile solo dal suo indirizzo. Le bozze stanno in `_drafts` e le vede solo chi ha fatto login.
- **Il nome del file .txt sceglie il template**: "The naming of these content files is important, because it defines which template is used to render the content". Se il template non esiste, Kirby usa `default.php`, che è l'unico obbligatorio.
- **Con più lingue** il file ha anche il codice della lingua: `about.it.txt`.
- **Formato del file**: un campo dopo l'altro, separati da `----`. Esempio reale:

```
Title: chi sono
----
Tagline: chi sono
----
Heading: Su di me
----
Quote: se non c'è movimento non c'è vita
```

getkirby.com/docs/guide/content/publishing-workflow · getkirby.com/docs/guide/content/creating-pages · getkirby.com/docs/guide/templates/basics

Il pannello

Il pannello (`/panel`) è l'interfaccia per chi modifica il sito. Mostra i campi descritti nelle **blueprint** (file YAML in `site/blueprints`) e scrive nei file .txt. Con Kirby 5 le modifiche non ancora pubblicate vengono salvate sul server, in una cartella `_changes` accanto ai contenuti, e si possono vedere in anteprima.

getkirby.com/releases/5.0

Utenti

Gli account del pannello sono file in `site/accounts`, esclusi da Git (gli Starterkit lo fanno già, e la guida sulla sicurezza lo raccomanda). La documentazione consiglia un account per persona. Il primo utente si crea con l'installazione del pannello, che di base non si può fare su un server pubblico (vedi capitolo 6).

getkirby.com/docs/guide/users/managing-users · getkirby.com/docs/guide/security

Lingue

Il multilingua si attiva con `'languages' => true` e un file per lingua in `site/languages`. La prima lingua deve essere quella principale. Ogni lingua ha i suoi file di contenuto e i suoi indirizzi; di base il codice della lingua entra nell'indirizzo (`/it/…`).

getkirby.com/docs/guide/languages/introduction

Cache

Kirby ha una cache delle pagine: si attiva in `config.php` con l'opzione `cache.pages` e salva i file in `site/cache`. Non mette in cache le richieste con parametri, quelle diverse da GET/HEAD e quelle che dipendono da sessione o cookie. **In questo progetto non è attivata.**

getkirby.com/docs/guide/cache

Miniature

Le misure ridotte delle immagini si chiedono nei template (`$image->resize(800)` , `$image->srcset('card')`) e Kirby le salva nella cartella `media` : "In a default setup, thumbnails are stored in the `/media` folder".

getkirby.com/docs/guide/files/resize-images-on-the-fly · getkirby.com/docs/cookbook/performance/responsive-images

Confronto con un CMS con database (per esempio WordPress)

Tema	Kirby (file)	CMS con database
Dove sono i contenuti	File .txt leggibili in `content/`	Tabelle in un database (MySQL)
Messa online	Si copiano le cartelle (FTP/SFTP); niente database da creare	File più database da creare e importare
Backup	Copia di cartelle (vedi capitolo 5)	File più esportazione del database
Versioni con Git	I contenuti sono file: si possono anche versionare	Il database va esportato a parte
Struttura del pannello	La decide lo sviluppatore con le blueprint	Di solito data dal CMS, estesa con plugin
Limiti	Pensato per siti di dimensione piccola e media; ricerche complesse su molti dati sono più lente	Adatto a molti dati e a ricerche complesse
Licenza	A pagamento, una per sito; gratis in locale	Spesso open source gratuito

La riga "Limiti" è un'opinione generale, non presa dalla documentazione di Kirby.

2. Le cartelle del progetto

Albero reale di `beweeg-physio-kirby`, solo le cartelle importanti:

```
beweeg-physio-kirby/
├── index.php            punto d'ingresso: carica kirby/bootstrap.php e mostra la pagina
├── .htaccess            regole di Apache: indirizzi puliti, cartelle protette
├── favicon.ico
├── composer.json/.lock  versione di Kirby richiesta (^5.2) e PHP supportati
├── content/             i contenuti (testi e foto), un .txt per pagina e lingua
│   ├── site.it.txt      Impostazioni sito (indirizzo, contatti, orari, slideshow)
│   ├── home/  1_chi-sono/  2_trattamenti/  3_servizio-a-domicilio/  4_prenota/  error/
├── site/
│   ├── blueprints/      campi del pannello (YAML): site, pages, fields, files, tabs, users
│   ├── templates/       un template PHP per tipo di pagina
│   ├── snippets/        pezzi riusabili: layout, header, menu, footer, picture, button…
│   ├── models/          treatment.php: metodo iconUrl() per le pagine trattamento
│   ├── plugins/         alt-obbligatorio, metodi-sito (nostri, nessun plugin esterno)
│   ├── config/          config.php (sempre) e config.localhost.php (solo in locale)
│   ├── languages/       it.php: lingua e testi dell'interfaccia (pulsanti, etichette)
│   ├── accounts/        utenti del pannello            → NON in Git
│   ├── sessions/        sessioni di login              → NON in Git
│   └── cache/           cache (es. UUID)               → NON in Git
├── assets/              css/main.css, js/main.js e slideshow.js, fonts/, icons/, images/
├── kirby/               il motore di Kirby (installato da Composer)
├── vendor/              librerie usate da Kirby (Composer)
├── media/               miniature e copie dei file, create da Kirby → NON in Git
├── scripts/             importa-contenuti.php: solo sviluppo, non si consegna
└── docs/                documenti di lavoro, non si consegnano
```

Una cartella `storage/` in questo progetto **non esiste**: account, sessioni e cache stanno nelle cartelle predefinite dentro `site/`.

Cosa va in Git e cosa no

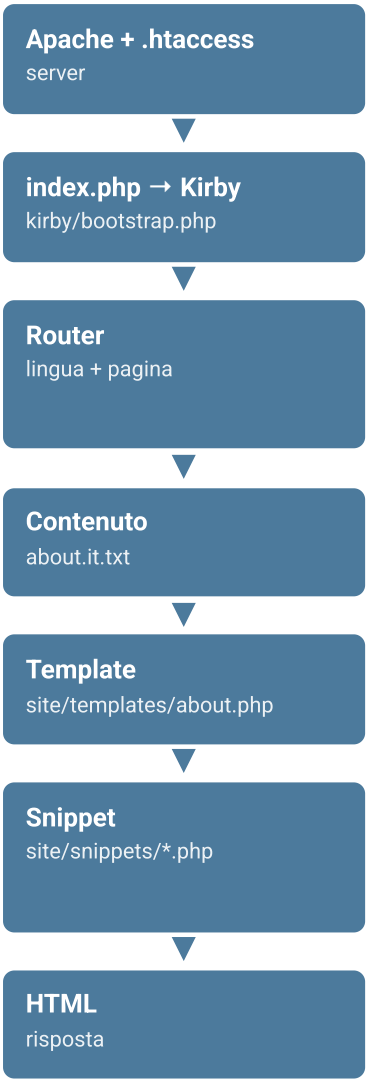
In Git	Fuori da Git (.gitignore)
codice (`site/` tranne account, sessioni e cache), `assets/`, `content/`, `kirby/`, `vendor/`, `composer.*`, `.htaccess`, `index.php`, `scripts/`, `docs/`	`site/accounts/`, `site/sessions/`, `site/cache/`, `media/`, `site/config/.license`, `.env` e `site/config/env.php`, `content/**/_changes/`, `docs/visual-originali/`, file degli editor

`kirby/` e `vendor/` sono nel repository di proposito: il sito si consegna via FTP così com'è, senza eseguire Composer sul server. `kirby/bootstrap.php` carica `vendor/autoload.php` nella radice del progetto (riga 30 del file), quindi `vendor/` serve davvero.

> **Da sapere:** la guida sulla sicurezza suggerisce di tenere fuori da Git anche `config.php`. Qui invece è in Git, perché contiene solo valori sicuri e nessun segreto; i segreti andrebbero in `env.php`, che è escluso.

3. Come funziona una richiesta

Esempio reale: qualcuno apre `https://…/it/chi-sono`.



Il file `chi-sono` non esiste su disco, quindi la regola finale dell'`.htaccess` passa la richiesta a `index.php`.

Kirby legge la configurazione (`config.php`, e in locale anche `config.localhost.php`), i plugin e le lingue.

`/it` = lingua italiana. `chi-sono` = la pagina con quell'indirizzo, cioè la cartella `content/1_chi-sono/`. Senza rotte personalizzate, "this folder creates a new URL".

Il nome del file (`about`) decide il template; `.it` è la lingua. I campi del file diventano metodi: `$page->heading()`, `$page->quote()`.

Divide il testo in due colonne, prepara foto e diplomi, poi apre lo snippet `layout` con gli slot.

`layout` → `header` (con `menu`) → `page-intro` (con `slideshow` e `picture`) → il contenuto del template (`section-heading`, `placeholder`, `button`) → `footer` (con `social` e `hours`).

La pagina finita. Le foto rimandano a miniature in `media/`, create la prima volta che servono.

getkirby.com/docs/guide/routing · getkirby.com/docs/guide/templates/basics · getkirby.com/docs/guide/templates/snippets

Dove sta ogni cosa, per /it/chi-sono

Cosa	File
Campi nel pannello	`site/blueprints/pages/about.yml`
Valori dei campi	`content/1_chi-sono/about.it.txt`
HTML della pagina	`site/templates/about.php`
Parti comuni	`site/snippets/layout.php`, `header.php`, `footer.php` …
Testi dell'interfaccia	`site/languages/it.php`, letti con `t('buttons.treatments')`
Indirizzo, orari, foto dello slideshow	`content/site.it.txt`, letti con `$site->…`
Stile e comportamento	`assets/css/main.css`, `assets/js/main.js`, `slideshow.js`

Gli snippet con gli slot

Ogni template si apre con `snippet('layout', slots: true)`: quello che scrive dopo finisce in `$slot`, dentro il `<main>` del layout. "If a snippet is still open at the end of the template, Kirby automatically closes it". Così head, header e footer stanno in un solo file.

getkirby.com/docs/guide/templates/snippets

4. Come è fatto questo sito

Blueprint e campi

Pagina	Blueprint / template	Campi
Impostazioni sito	`site.yml`	Scheda "Pagine" + scheda "Impostazioni sito": via, NPA, località, cantone, telefono, email, WhatsApp, Facebook, Instagram, orari (tabella), link di prenotazione, foto dello slideshow. Indirizzo, numeri e link hanno `translate: false`.
Home	`home.yml` / `home.php`	sottotitolo; chi sono (titolo, paragrafo guida, testo); numeri (max 3); cosa faccio (titolo); un plus (titolo, paragrafo guida, testo); dove trovi lo studio (titolo); scheda SEO.
Chi sono	`about.yml` / `about.php`	sottotitolo, titolo, testo, citazione, firma, foto di Jan (max 2), titolo diplomi, diplomi (la didascalia è nella foto), SEO.
Trattamenti	`treatments.yml` / `treatments.php`	sottotitolo, titolo, titolo "altri trattamenti", elenco delle sottopagine (ordinabile, niente "aggiungi"), SEO.
Trattamento (×4)	`treatment.yml` / `treatment.php`	nome breve, sottotitolo, paragrafo guida, testo, foto (solo per le schede). L'icona è un file scelto dal modello `TreatmentPage::iconUrl()`.
Servizio a domicilio	`home-visits.yml` / `home-visits.php`	sottotitolo, titolo, sottotitolo della sezione, paragrafo guida, testo, SEO.
Prenota	`book.yml` / `book.php`	sottotitolo, titolo, introduzione, testo se la prenotazione online non è attiva, SEO.
Errore 404	`error.yml` / `error.php`	nascosta nel pannello (`read: false`); i testi sono in `it.php`.

Campi condivisi con `extends`: `fields/lead.yml` e `fields/text.yml` (writer con solo grassetto, corsivo e link), `fields/images.yml` (campo foto), `tabs/seo.yml`, `files/image.yml` (testo alternativo obbligatorio, didascalia; solo JPEG, PNG, WebP). Le pagine fisse non si possono cancellare né rinominare (`options: delete, changeSlug, changeTemplate: false`).

getkirby.com/docs/reference/panel/fields/writer · …/fields/files · …/fields/structure · …/blueprints/file

Contenuti importati con lo script

`php scripts/importa-contenuti.php` legge i testi del vecchio progetto (`../beweeg-physio/content/it.json` e `lib/site.ts`) e li scrive con l'API di Kirby (`update()`, `createFile()`):

- converte `**grassetto**` in `<strong>`;
- copia le 8 foto;
- scarta le bozze `_changes` delle pagine che aggiorna;
- alla fine elenca le chiavi del JSON rimaste fuori;
- con `en` o `de` importa testi, titoli e indirizzi tradotti (serve prima il file della lingua).

È solo per lo sviluppo: rilanciarlo sovrascrive i campi.

Layout e snippet riusabili

- `layout`: head (titolo da "Titolo per Google" o "pagina · sito", `noindex`, favicon, preload del font), skip link, header, main, footer.

- `page-intro` + `slideshow` + `picture`: l'apertura di ogni pagina. "Beweeg physio" è H1 solo nella home; nelle altre pagine l'H1 è il titolo del contenuto.
- `section-heading`, `lead-text`, `button` (anche come `<span>` dentro una scheda, o con `newTab`), `band`, `treatment-card`, `treatment-link`, `placeholder`, `map`, `hours`, `social`, `language-switch`.
- **Campi writer:** si stampano così come sono (`<?= $page->text() ?>`), come dice la documentazione. Kirby li ripulisce al salvataggio: in una prova, uno `<script>` è stato tolto anche salvando con `update()`.

Menu

- Le voci sono `$site->children()->listed()`, nell'ordine dei numeri delle cartelle.
- `aria-current="page"` sulla pagina aperta (`isActive()`); `"true"` sul genitore di una sottopagina (`isOpen()`, che dal codice sorgente vale per la pagina corrente e per i suoi genitori).
- **Grassetto senza spostamenti:** ogni link ha `data-label`, e un `::after` invisibile in grassetto riserva la larghezza.
- **Mobile:** `main.js` (caricato nel `<head>` senza `defer`) aggiunge subito la classe `js`; il menu diventa un pannello a tutto schermo. Il resto della pagina diventa `inert`, Esc chiude il menu e il focus torna al pulsante. Senza JS il menu resta visibile sotto il logo.
- **Header sticky** a tutte le larghezze, con `scroll-padding-top` per non coprire il focus.

Slideshow

- Senza JS si vede solo la prima foto e i comandi hanno `hidden`.
- `slideshow.js` (caricato solo se le foto sono almeno 2) cambia foto ogni 6 secondi e si ferma con il pulsante pausa, con il mouse o con il focus nell'apertura.
- Con "riduci movimento" attivo non parte da solo.
- I comandi stanno in una striscia riservata, quindi non coprono il testo.

Immagini e WebP

- In `config.php` ci sono i preset `thumbs.srcsets`: `hero` (640–2560 px) e `card` (400–1200 px), più le versioni `-webp`.
- Lo snippet `picture` stampa una `<source>` WebP solo se `option('beweeg.webp')` è vero. Questa opzione è calcolata da `function_exists('imagewebp')` e da `gd_info()`: se il server non sa creare WebP, si usa solo JPEG. È stato provato simulando un server senza WebP.
- `width` e `height` vengono da `$image->resize(…)`, quindi la pagina non salta mentre le foto si caricano.
- Il testo alternativo è obbligatorio. Il plugin `alt-obbligatorio` (hook `page.update:before` e `site.update:before`) blocca il salvataggio di una pagina se una sua foto non ce l'ha, perché il campo obbligatorio da solo non blocca il caricamento.

Orari

Una tabella (structure) nelle Impostazioni sito, con una riga per gruppo di giorni:

- "Giorni (come appare sul sito)", per esempio "lu–ma e gi–ve";
- caselle da lunedì a domenica;
- una tabella interna "Fasce orarie" (Apre e Chiude, campi ora salvati come `08:00:00`).

Lo snippet `hours` mostra "08:00 – 12:00" con `toDate('H:i')`; senza fasce mostra `t('hours.closed')`. Caselle e ore serviranno anche ai dati strutturati per Google.

404

`content/error/` + `templates/error.php`. Kirby risponde con lo stato 404 (verificato); testi e pulsante vengono da `it.php`.

Configurazione per sviluppo e produzione

`config.php` vale ovunque e contiene solo valori sicuri per la produzione (`debug => false`).
`config.localhost.php` si aggiunge solo quando l'host è `localhost` (anche con la porta 8000, verificato) e accende il debug. "the settings in the standard `config.php` file are always used".

getkirby.com/docs/guide/configuration · getkirby.com/docs/reference/system/options/debug

Multilingua con il solo italiano

- `'languages' => true` e `'languages.detect' => true`; `site/languages/it.php` senza `url`, quindi gli indirizzi iniziano con `/it` e la radice `/` porta a `/it`.
- I file di contenuto sono `*.it.txt`.
- I testi dell'interfaccia sono in `translations` e si leggono con `t()`; si possono modificare anche dal pannello, nell'editor delle lingue.
- Il selettore lingua non si vede finché c'è una lingua sola.

getkirby.com/docs/guide/languages/introduction · …/languages/custom-language-variables

5. Come si fa

Cambiare un testo

Dal pannello: apri la pagina, modifica il campo, salva. Per i testi dell'interfaccia (pulsanti, "chiuso", etichette): pannello → Lingue → Italiano, oppure direttamente `site/languages/it.php`. Una modifica dal pannello riscrive quel file sul server: quindi va salvato anche nei backup.

Aggiungere un campo

1. Aggiungilo alla blueprint (per esempio `site/blueprints/pages/about.yml`), con etichetta in italiano e, se serve, un aiuto.
2. Stampalo nel template: `<?= $page->nomecampo()->esc() ?>` per il testo semplice, senza `esc()` se è un writer.
3. Gestisci il campo vuoto: `isNotEmpty()` prima di stampare l'elemento.

Aggiungere una pagina

1. Blueprint `site/blueprints/pages/nome.yml` e template `site/templates/nome.php`, che apre `snippet('layout', slots: true)`.
2. Crea la cartella in `content/` con il numero per l'ordine nel menu (es. `5_nuova/nome.it.txt`). Dal pannello oggi non si può, perché in `site.yml` la sezione delle pagine ha `create: false`.
3. Il menu la mostra da solo se è "listed".

Aggiungere una lingua (es. inglese)

1. Crea `site/languages/en.php` (`code`, `name`, `locale`, `direction`, `translations` con le stesse chiavi di `it.php`). Si può fare anche dal pannello.
2. Lancia `php scripts/importa-contenuti.php en`: titoli, testi e indirizzi tradotti.
3. Controlla nel pannello i campi con `translate: false` (indirizzo, link: sola lettura nelle altre lingue) e la tabella degli orari, che si traduce per intero.
4. Il selettore lingua compare da solo con due lingue.

Aggiornare Kirby con Composer

```
composer update getkirby/cms     # in locale, mai sul server
php -S localhost:8000 kirby/router.php   # prova tutte le pagine e il pannello
git add kirby vendor composer.lock && git commit
```

"Kirby however is automatically installed in the `kirby` directory". Le versioni minori di solito non richiedono modifiche; una versione maggiore può richiedere di aggiornare il codice. La licenza copre le versioni uscite nei 3 anni successivi all'attivazione.

getkirby.com/docs/cookbook/setup/composer · getkirby.com/license

Mettere il sito sul server con SFTP

1. Prepara il pacchetto: `index.php`, `.htaccess`, `favicon.ico`, `assets/`, `content/`, `kirby/`, `site/` (senza `accounts/`, `sessions/`, `cache/`), `vendor/`.
2. Non caricare: `.git`, `docs/`, `scripts/`, `CLAUDE.md`, `media/`, file `.env`, `config.localhost.php` (innocuo, ma non serve).
3. Nel programma SFTP attiva i file nascosti: "make sure to copy the `.htaccess` file as well".

4. Prova tutte le pagine, il pannello e il caricamento di una foto.

getkirby.com/docs/guide/quickstart

Fare un backup

Da copiare dal server: `content/` (testi e foto), `site/accounts/` (utenti), `site/languages/` (se i testi dell'interfaccia sono stati cambiati dal pannello), la licenza in `site/config/`. Il resto è nel repository o si rigenera (`media/`, `site/cache/`). La guida sulla sicurezza chiede di avere sempre un backup prima di un aggiornamento.

Creare l'utente in produzione

Di base il pannello non si installa su un server pubblico: "As a security measure, you cannot install the Panel on a public server by default". Due strade documentate:

- creare l'account amministratore prima di pubblicare (la guida sulla sicurezza lo raccomanda);
- attivare per poco `'panel' => ['install' => true]`, creare l'utente con una password nuova e lunga, poi togliere l'opzione: "It's recommended to remove this setting after the installation".

getkirby.com/docs/reference/system/options/panel · getkirby.com/docs/guide/security

6. Sicurezza e produzione

.htaccess

Oggi è quello dello Starterkit:

- blocca i file che iniziano con il punto (es. `.git`), tranne `.well-known`;
- blocca l'accesso diretto a `content/`, `site/` e `kirby/`;
- manda tutto il resto a `index.php`, comprime le risposte e aggiunge `X-Content-Type-Options: nosniff`.

Sono le protezioni che la guida sulla sicurezza indica quando non si può mettere il codice fuori dalla cartella pubblica.

> **Ancora da fare (passo K7):** redirect a HTTPS, intestazioni di sicurezza (HSTS, referrer, CSP con `frame-src` per Google Maps e lo strumento di prenotazione), `Options -Indexes` per non mostrare l'elenco dei file.

Permessi dei file

La documentazione dice di non usare permessi troppo larghi e cita esplicitamente "777"; per i valori rimanda a guide esterne. Kirby deve poter scrivere in `content/`, `media/` e `site/` (account, sessioni, cache, lingue). I valori esatti per Hostpoint sono in "Da verificare".

Licenza

- Una per sito: "Each project (defined by its URL) needs its own license", e ogni sottodominio o cartella conta come sito a sé.
- In locale è gratis.
- Va attivata dal pannello (o dal sito di Kirby) appena il sito è pubblicato.
- Include le versioni uscite nei 3 anni successivi.

getkirby.com/license

Debug

Spento in `config.php`, acceso solo da `config.localhost.php`. "Make sure to disable `debug` mode in production!": mostrerebbe a chiunque dettagli del server.

Pannello

- Solo con HTTPS: senza, password e dati viaggiano in chiaro.
- Kirby blocca un IP o un utente dopo 10 tentativi sbagliati in un'ora (configurabile con le opzioni `auth`).
- Un account per persona, con password lunghe.
- Si può cambiare l'indirizzo del pannello (`panel.slug`) o spegnerlo (`'panel' => false`).

Cosa non caricare sul server

`.git`, `docs/`, `scripts/`, `CLAUDE.md`, file `.env`, account e sessioni locali, `site/cache/`, `media/` (si rigenera), la licenza locale. Lo `scripts/importa-contenuti.php` rifiuta di girare dal browser, ma sul server non deve proprio esserci.

Indicizzazione

Oggi `layout.php` stampa sempre `<meta name="robots" content="noindex, nofollow">`: il sito resta fuori da Google fino all'ok del cliente. La gestione con un'opzione unica è prevista nel passo K5.

7. Storia e motivi delle scelte

Perché solo Kirby

- **La demo in Next.js** (cartella `../beweeg-physio`) è servita come anteprima grafica. Ha richiesto Node e un hosting diverso da un PHP standard: non poteva girare sul server di Sara.
- **Kirby gira su un hosting PHP normale**, si consegna via FTP e ha un pannello che chi cura la comunicazione conosce già.
- Un'integrazione Next + Kirby "headless" avrebbe voluto dire due sistemi da tenere allineati. Il progetto Next resta come riferimento in sola lettura.

Perché CSS semplice senza build

- Un solo `main.css` con variabili: si consegna così com'è, senza Node né compilazione, e lo può leggere chiunque.
- I valori vengono dal visual e dall'audit tipografico; i componenti usano solo variabili.
- Niente stili nell'HTML (nemmeno `style="…"`): una CSP futura potrà vietare gli stili inline.

Perché il campo writer e non i blocchi

- Il visual è definitivo: ogni pagina ha sezioni fisse. Con i blocchi o i layout di Kirby chi modifica potrebbe cambiare la struttura che il cliente ha approvato.
- Il writer limitato a grassetto, corsivo e link basta per i testi reali, non richiede di conoscere il markdown e produce HTML pulito che si stampa così com'è.
- Se in futuro serve "cambiare il layout", si possono aggiungere blocchi definiti da noi.

Perché la tabella degli orari è fatta così

- Prima c'erano due fonti: il testo del footer ("lu–ma e gi–ve" con "08:00 – 12:00") e i dati per Google (giorni in inglese e ore).
- Ora si scrive tutto una volta, nello stesso posto: il testo dei giorni come appare, le caselle dei giorni e le fasce come ore vere.
- Il footer usa testo e ore; i dati per Google useranno caselle e ore.
- **Limite noto:** la tabella si traduce per intero, quindi in ogni lingua giorni e ore sono una copia.

8. Domande che potrebbero farti

1. Perché gli snippet con gli slot e non header.php + footer.php?

Con gli slot head, header e footer stanno in un solo file (`layout.php`), che si apre e si chiude per intero; il template riempie solo `$slot` . È una funzione documentata di Kirby.

2. Perché `vendor/` e `kirby/` sono nel repository?

Il sito si consegna via FTP senza eseguire Composer sul server. `kirby/bootstrap.php` carica `vendor/autoload.php` della radice, quindi senza `vendor/` il sito non parte. Il prezzo è un repository più pesante (circa 11 MB).

3. Perché le cartelle `_changes` sono ignorate da Git?

Sono le modifiche non ancora pubblicate dal pannello (Kirby 5): bozze di lavoro di una persona, non contenuto approvato. Nel repository va solo il contenuto pubblicato.

4. Perché non usi KQL?

KQL è un plugin ufficiale per interrogare i contenuti via API, utile se il sito è "headless" (un front-end separato). Qui Kirby genera le pagine da solo con i template, quindi non serve. Era nel piano iniziale con Next, abbandonato.

5. Come gestisci la cache?

La cache delle pagine non è attiva. Il sito è piccolo e non l'ho misurato: si può attivare con `cache.pages` quando ci sarà il server vero, provando che le pagine cambiate dal pannello si aggiornino. Kirby usa comunque `site/cache` per i propri dati (es. UUID).

6. E le miniature?

Preset `srcset` in `config.php` ; Kirby le crea in `media/` quando servono. `media/` è fuori da Git e non si carica sul server. WebP solo se GD lo supporta.

7. Come aggiungi un'altra lingua?

File `site/languages/en.php` , poi lo script di import con `en` per titoli, testi e indirizzi tradotti; i testi dell'interfaccia vanno nelle `translations` . Il selettore compare da solo. Da controllare a mano: campi `translate: false` e tabella degli orari.

8. Cosa succede a `content/` quando aggiorni Kirby?

`composer update getkirby/cms` aggiorna `kirby/` (e le librerie in `vendor/`); `content/` e `site/` sono file nostri che Composer non gestisce. Aggiorno sempre in locale, con backup, e poi carico solo `kirby/` e `vendor/` .

9. Come proteggi il pannello?

HTTPS obbligatorio, un account per persona con password lunghe, il blocco di Kirby dopo 10 tentativi falliti in un'ora, installazione del pannello sul server solo per il tempo necessario, debug spento. Si può anche cambiare l'indirizzo `/panel` .

10. Perché writer e non il campo blocks o layout?

Il visual approvato ha sezioni fisse; un writer limitato non lascia rompere la grafica e non richiede markdown. Vedi capitolo 7.

11. Perché i testi dei pulsanti sono in `it.php` e non in campi?

Sono testi dell'interfaccia, uguali in tutte le pagine: con `t()` si traducono una volta per lingua e si modificano dall'editor delle lingue nel pannello.

12. Il testo alternativo obbligatorio non bastava come campo `required`?

No: Kirby lo controlla solo quando si salva la foto, non quando la si carica, quindi una foto senza testo alternativo poteva finire in pagina. Il plugin blocca il salvataggio della pagina finché la foto non ce l'ha. È stato provato passaggio per passaggio.

13. Perché un modello per le icone dei trattamenti?

Le icone sono file del tema, non contenuti. `TreatmentPage::iconUrl()` tiene la corrispondenza "cartella → icona" in un solo posto e usa `uid()`, che non cambia quando l'indirizzo viene tradotto.

14. Perché un plugin per `phoneLink()`?

Footer e Prenota devono trasformare il numero allo stesso modo. Kirby non ha modelli per `$site`: il modo documentato per aggiungergli un metodo è l'estensione `siteMethods` in un plugin.

15. Perché dal pannello non si possono creare o cancellare pagine?

Il sito ha pagine fisse con indirizzi pubblici e template dedicati; un trattamento nuovo richiede anche la sua icona. Le opzioni delle blueprint lo impediscono per evitare indirizzi rotti per errore.

16. Perché il pannello non accetta SVG?

Un SVG può contenere codice. La blueprint delle immagini accetta solo JPEG, PNG e WebP; le icone SVG sono file del tema in `assets/`.

17. Come si separa la configurazione locale da quella del server?

`config.php` sicuro per la produzione, `config.localhost.php` letto in più solo in locale. Per segreti o valori del server si può usare `env.php`, escluso da Git.

18. Perché il JavaScript del menu è nel <head> senza defer?

La prima riga aggiunge la classe `js` prima che la pagina venga disegnata: così su mobile il menu si chiude dietro al pulsante senza lampeggiare. Il file è piccolo e locale; il resto aspetta `DOMContentLoaded`.

19. Lo script di import si può usare sul sito pubblicato?

No. Sovrascrive i campi con i testi del vecchio progetto: serve solo in locale. Rifiuta di girare dal browser e non fa parte della consegna.

20. Perché la mappa è un iframe senza chiave API?

Era la richiesta: niente chiavi da gestire. L'indirizzo usato (`maps?q=…&output=embed`) però Google non lo documenta, e la mappa carica cookie di Google (da indicare nella privacy). L'alternativa è un'immagine statica, come nel visual.

9. Cose che non so o da verificare

- **Permessi dei file sul server:** la documentazione non dà valori, dice solo di non usare 777. Da chiedere a Hostpoint o provare (di solito 755 per le cartelle, 644 per i file: non verificato).
- **Driver delle miniature:** la documentazione mostra come usare ImageMagick ma, nelle pagine lette, non dice qual è il predefinito. Che sia GD l'ho dedotto dal funzionamento in locale.
- **Cancellare `media/`:** in locale, dopo averla svuotata, le miniature si sono ricreate da sole, ma la documentazione non lo dice esplicitamente.
- **Cache delle pagine:** la documentazione non dice esplicitamente che di base è spenta, né come si svuota quando cambia un contenuto. Da leggere meglio prima di attivarla.
- **Licenza:** che il file si chiami `site/config/.license` viene dal `.gitignore` dello Starterkit, non dalla pagina della licenza. Da verificare con la prima attivazione.
- **Backup:** la pagina della documentazione sui backup che ho cercato non esiste (404). L'elenco del capitolo 5 viene dalla struttura del progetto, non da una raccomandazione ufficiale.
- **Bozze `_changes`:** so che esistono e dove stanno (note di rilascio di Kirby 5), ma non ho letto la guida completa sul flusso "salva / pubblica" del pannello, né come si chiamano esattamente i pulsanti.
- **Campi non tradotti:** la documentazione parla di "fallback" alla lingua principale solo in modo indiretto. Da provare quando si aggiunge l'inglese.
- **Hostpoint:** versione di PHP, supporto WebP in GD, `mod_rewrite` e `mod_headers` attivi, accesso SFTP, backup automatici. Tutto da verificare sul server (c'è una mini-guida per Sara).
- **Doppia autenticazione nel pannello:** non ho verificato se Kirby 5 la offre e come si attiva.
- **Cartella `storage/`:** non esiste in questo progetto; non ho verificato se Kirby 5 preveda un modo per spostare account, sessioni e cache fuori da `site/`.
- **Embed di Google Maps senza chiave:** funziona oggi, ma non è un'interfaccia documentata da Google e potrebbe cambiare.
- **Safari su iPhone:** header sticky, menu e slideshow sono stati provati solo in Chrome.
- **Licenza di Kirby:** prezzi e varianti (Basic, Enterprise) non sono nella pagina letta; vanno controllati sulla pagina d'acquisto al momento di comprarla.